2026-06-08 - Power Management and Sleep

Goal

Configure Zephyr's power management subsystem to put the nRF52840 into low-power sleep states between tasks, dramatically reducing current consumption.

What you will learn

- Zephyr's power management states and how the PM subsystem works
- How to enable automatic idle sleep with `CONFIG_PM`
- How to use `k_sleep` and `k_msleep` to hint the kernel to sleep
- How to measure current draw with and without PM enabled
- How to use the nRF52840's System OFF mode for ultra-low power

Overview

The nRF52840 has several power states: | State | Current | Wake source | |-------|---------|-------------| | Active (CPU running) | ~4 mA | — | | WFI (Wait For Interrupt) | ~1.5 mA | Any IRQ | | System ON sleep (radio off) | ~2–4 µA | GPIO, timer, RTC | | System OFF | ~0.4 µA | GPIO DETECT, RESETN |

Zephyr's PM subsystem selects the deepest safe sleep state automatically whenever all threads are blocked or sleeping.

prj.conf — enable automatic power management

```
CONFIG_PM=y
CONFIG_PM_DEVICE=y
CONFIG_SERIAL=n          # disable UART to allow deeper sleep
CONFIG_LOG=n             # logging keeps UART active
```

> **Note:** Disabling UART removes the console. Use RTT (Real-Time Transfer) for debug output in low-power builds:
>
> ```
> CONFIG_USE_SEGGER_RTT=y
> CONFIG_RTT_CONSOLE=y
> ```

How automatic sleep works

When all Zephyr threads call `k_msleep()` or are blocked on semaphores, the kernel calculates how long until the next wake event, then calls `pm_state_force()` or lets the idle thread select a sleep state. No application code change is needed — just enable `CONFIG_PM=y`.

Entering System OFF (deepest sleep)

System OFF is a full power-down. The CPU does not resume — it resets on wake. Use this for ultra-low power when long intervals (minutes/hours) are acceptable.

```
#include <zephyr/pm/pm.h>
#include <hal/nrf_gpio.h>

void enter_system_off(void)
{
    /* Configure a GPIO pin as wake source */
    nrf_gpio_cfg_sense_input(NRF_GPIO_PIN_MAP(0, 11),
                             NRF_GPIO_PIN_PULLUP,
                             NRF_GPIO_PIN_SENSE_LOW);

    /* Enter System OFF — code here does not return */
    pm_state_force(0u, &(struct pm_state_info){PM_STATE_SOFT_OFF, 0, 0});
    k_sleep(K_FOREVER);
}
```

Wake-on-motion pattern (with KXTJ3 INT pin)

```
1. Read + process accelerometer data
2. Configure KXTJ3 wake-on-motion interrupt
3. Enter System OFF sleep
4. KXTJ3 INT pin goes high → nRF52840 resets
5. main() runs again → read + process → sleep
```

Measuring power savings

Use a Nordic PPK2 (Power Profiler Kit 2) or a µCurrent meter in series with the VDD supply. Compare: - No sleep: ~4 mA continuous - `CONFIG_PM=y` with 1 s sleep: ~8 µA average - System OFF with GPIO wake: ~0.5 µA average

Keeping peripherals active during sleep

Some drivers support `PM_DEVICE` — they can be suspended/resumed automatically. Enable with:

```
CONFIG_PM_DEVICE=y
CONFIG_PM_DEVICE_RUNTIME=y
```

Why this matters

Battery life is a critical spec for wearables and IoT sensors. A coin cell (CR2032, 220 mAh) lasts hours at 4 mA but over a year at 8 µA average.

Practice tasks

1. Enable `CONFIG_PM=y` and measure current during `k_msleep(5000)`.
2. Compare against the same build without `CONFIG_PM`.
3. Implement System OFF with button (sw0) as wake source — confirm it wakes and boot-counts (NVS from Day 22) increments on each wake.

Example folder

See `src/2026-06-08_Power_Management_Sleep` for the day's code.

Next topic

You now have all the core Zephyr building blocks. Advanced topics: MCUboot OTA updates, BLE GATT services, LwM2M cloud integration.